Prosta gra od pierwszych lekcji — JAVA

24 lekcje. Cel: samodzielnie zbudowana arena z ruchem, walką, rozwojem, interfejsem i restartem.

01. Scena, obiekty i komponenty
02. Sterowanie i kierunek postaci
03. Collidery, przeszkody i granice świata
04. Pierwszy przeciwnik i pościg
05. Pociski, HP i pierwsza pętla walki
06. Własna klasa komponentu i cykl życia
07. Czas, wektory i stała prędkość
08. Sprite, kafelki, odbicie i warstwy
09. Fale, konfiguracja i powtarzalny spawn
10. Health, obrażenia i nietykalność gracza
11. Weapon, cooldown i automatyczny atak
12. Triggery, monety i własny Wallet
13. Śmierć przeciwnika, łup i doświadczenie
14. AI: wykrywanie, flankowanie i ucieczka
15. Kamera i przestrzenie współrzędnych
16. Canvas, ręczne rysowanie i HUD
17. Pauza i działające menu
18. Wybór ulepszenia i własne reguły zakupów
19. Feedback trafienia, tweeny i floating text
20. Pociski naprowadzane i przebijające
21. Kompozycja, fabryki i wyposażenie
22. Start, przegrana, zwycięstwo i restart
23. Wydajność, testowanie i zapis projektu
24. Projekt końcowy: Łąka przetrwania

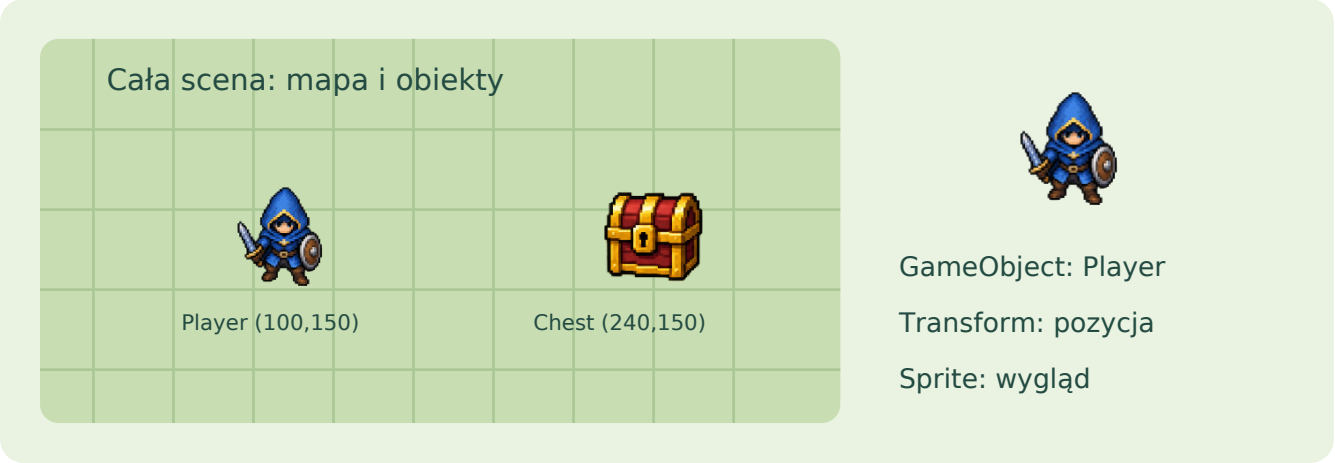
Każda lekcja: 1. mała zmiana z przykładem; 2. podobny problem samodzielnie; 3. samodzielna rozbudowa. Złoty kolor pokazuje kod odpowiedzialny za zmianę.

Pierwszych pięć lekcji daje ruch, przeszkody, przeciwnika i walkę z HP. Gotowe Health i Weapon rozbudujesz w lekcjach 10 i 11; AI w 14. Poniżej materiały i zadania startowe.

Lekcja 1 · Scena, obiekty i komponenty

Czego się nauczysz

Rozróżniasz obiekt, jego Transform i komponent wyglądu. Tworzysz dwie niezależne instancje i odczytujesz pozycję obiektu na mapie.



Z czego składa się scena?

Game przechowuje całą mapę i obiekty. GameObject jest kontenerem, Transform określa pozycję, a komponenty nadają mu wygląd i zachowania. Sprite lub ShapeRenderer rysuje obiekt; sam GameObject jest niewidoczny.

Gdzie znajduje się obiekt?

Transform określa punkt obiektu w świecie. W tym silniku X rośnie w prawo, Y w dół. Przykład (100,150) oznacza 100 jednostek w prawo i 150 w dół od początku mapy.

Komponenty definiują obiekt

Sprite i collider umieszczamy względem pozycji obiektu. Ich wymiary oraz przesunięcia są osobne. Transform nie określa środka widocznych pikseli grafiki.

Gotowa baza i dalsza nauka

Gotowa baza: infrastruktura i mechaniki jeszcze niewyjaśnione w kursie. Edytuj wyłącznie miejsce TODO; pliki pomocnicze możesz obejrzeć.

Lekcja 1 · zadania

1. Z przykładem — Pierwszy obiekt

Masz gotowy Player w (80,90). Dodaj mu tylko ShapeRenderer 40×30 w kolorze #76b9f2. Uruchom: niewidoczny dotąd obiekt stanie się widoczny.

Zmiana, która uruchamia mechanikę:

GameMain.java

```
player.addComponent(new ShapeRenderer(40, 30, "#76b9f2"));
```

GameObject już istnieje i ma pozycję. Dodanie ShapeRenderer nadaje mu widoczny wygląd. Komponenty określają, co obiekt potrafi: renderer rysuje, kontroler porusza, collider wykrywa kontakt.

- Istnieje jeden aktywny Player z własnym Transform i ShapeRenderer.
- Obraz prostokąta obejmuje x=60..100 i y=75..105; pozycja obiektu pozostaje (80,90).
- Po 20 aktualizacjach liczba obiektów nadal wynosi 1 i pozycja się nie zmienia.

2. Samodzielnie — Dwie osobne platformy

Masz dwie widoczne platformy. Samodzielnie dopisz przesunięcie tylko PlatformB o 40 px w prawo. PlatformA ma pozostać bez zmian.

- Istnieją dwa różne obiekty i dwa różne Transform.
- PlatformA ma środek (120,180) i granice x=70..170; PlatformB ma środek (340,120) i granice x=300..380.
- Zmiana koloru lub pozycji PlatformB nie zmienia PlatformA.

3. Rozbudowa — Mała arena

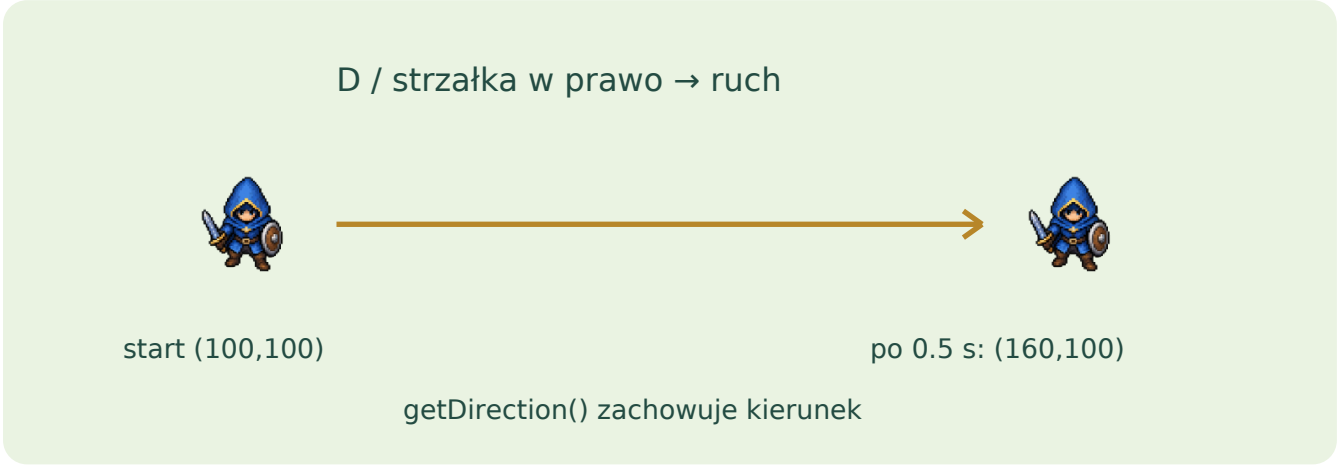
Gracz i pozycje obiektów są gotowe. Samodzielnie dodaj Enemy wygląd slime 32×32, a Coin wygląd coin 16×16. Nie twórz kolejnych obiektów.

- Scena zawiera dokładnie trzy wskazane obiekty i odpowiadające im tekstury.
- Player zajmuje x=84..116,y=84..116; Coin x=172..188,y=172..188.
- Zmiana pozycji Coin na (200,180) nie przesuwa gracza ani przeciwnika.
- Po sekundzie sceny bez zachowań pozycje pozostają stałe.

Lekcja 2 · Sterowanie i kierunek postaci

Czego się nauczysz

Rozróżniasz przytrzymanie klawisza i pojedyncze naciśnięcie. Oddzielasz odczyt input od wykonania ruchu przez CharacterController2D. Odczytujesz ostatni niezerowy kierunek z CharacterController2D.getDirection(), także po zatrzymaniu postaci.



Od klawisza do ruchu

PlayerController jest klasą ucznia. Odczytuje wejście i wybiera kierunek. CharacterController2D wykonuje ruch z uwzględnieniem czasu i przeszkód.

Przytrzymanie czy nowe naciśnięcie?

Przytrzymanie służy do chodzenia. Nowe naciśnięcie wywołuje pojedynczą akcję. Trzy klatki przytrzymania nie powinny oznaczać trzech nowych naciśnięć.

Kierunek zostaje po zatrzymaniu

CharacterController2D.getDirection() zwraca kopię ostatniego niezerowego kierunku. Po ruchu w prawo i puszczeniu D gracz stoi, ale kierunek nadal wynosi (1,0). Broń w lekcji 5 użyje go do strzału.

Gotowa baza i dalsza nauka

Gotowa baza: infrastruktura i mechaniki jeszcze niewyjaśnione w kursie. Edytuj wyłącznie miejsce TODO; pliki pomocnicze możesz obejrzeć.

Lekcja 2 · zadania

1. Z przykładem — Ruch WASD i strzałkami

Gracz, kontroler i odczyt WASD/strzałek są gotowe. Dopisz jedno wywołanie move(x, y, speed) w PlayerController. Po uruchomieniu D ma przesuwać gracza w prawo, a brak klawiszy zatrzymywać.

Zmiana, która uruchamia mechanikę:

PlayerController.java

```
requireComponent(CharacterController2D.class).move(x, y, speed);
```

Gotowy kod odczytuje WASD i strzałki. Jedno wywołanie move przekazuje kierunek i prędkość kontrolerowi. Kontroler uwzględnia czas oraz normalizuje przekątną — nie mnożymy speed ponownie przez delta.

- D trzymane 0.5 s przesuwa Player do (160,100); ArrowRight daje identyczny wynik.
- Przez 0.5 s bez wejścia pozycja pozostaje (100,100).
- A+D oraz W+S dają zerowy ruch na danej osi.
- S+D przez sekundę daje dystans 120 px, z zachowaniem równych składowych.

2. Samodzielnie — Sprint

Gotowy gracz porusza się z prędkością 100. Samodzielnie zmień parametr move: Shift daje 200 px/s, bez Shift 100. Nie kopiuj logiki kierunku.

- D+Shift przez 0.5 s ze startu x=100 daje x=200.
- D przez kolejne 0.5 s po puszczeniu Shift daje x=250.
- W+D+Shift przez 0.5 s daje dystans 100 px.
- Sam Shift nie porusza postaci.

3. Rozbudowa — Jedna akcja na naciśnięcie

Ruch jest gotowy. Uzupełnij ActionCounter tak, aby jedno nowe naciśnięcie Space zwiększało actions raz. Przytrzymanie nie może naliczać kolejnych akcji.

- Trzymanie Space przez 10 kolejnych aktualizacji daje actions=1.
- Puszczenie na jedną aktualizację i ponowne naciśnięcie daje actions=2.
- Działa jednoczesny ruch z D i akcja Space; prędkość pozostaje 120 px/s.
- Ponowne wejście focus do planszy nie tworzy fikcyjnego naciśnięcia ani zablokowanego klawisza.

Lekcja 3 · Collidery, przeszkody i granice świata

Czego się nauczysz

Odróżniasz renderer od fizycznego kształtu i triggera. Używasz kontrolera do blokowania ruchu i ślizgania przy przeszkodach.

Wygląd ściany + collider = przeszkoda



Ściana: środek x=200

Lewa krawędź x=190

Gracz: połowa szerokości 16

Stop: x=190−16=174

Obraz i fizyczny kształt

Sprite jest grafiką, collider opisuje obszar kontaktu. Koło może być mniejsze od sprite’a. Oba kształty mają własne wymiary.

Ruch zatrzymuje się przy ścianie

CharacterController2D sprawdza przeszkodę podczas ruchu. Bezpośrednie setPosition zmienia pozycję, ale nie zastępuje ruchu z kolizjami.

Ruch wzdłuż przeszkody

Przy kontakcie kontroler może zablokować składową skierowaną w ścianę i pozostawić ruch wzdłuż niej. Sprawdź kształty przez opcję Collidery.

Gotowa baza i dalsza nauka

Gotowa baza: infrastruktura i mechaniki jeszcze niewyjaśnione w kursie. Edytuj wyłącznie miejsce TODO; pliki pomocnicze możesz obejrzeć.

Lekcja 3 · zadania

1. Z przykładem — Ściana zatrzymuje gracza

Gracz i widoczna ściana są gotowe. Dodaj ścianie tylko Collider2D(20,200). Uruchom i przytrzymaj D: gracz ma zatrzymać się przed ścianą na x=174.

Zmiana, która uruchamia mechanikę:

GameMain.java

```
wall.addComponent(new Collider2D(20,200));
```

Widoczna ściana sama nie blokuje ruchu. Dodanie Collider2D o wymiarach 20×200 daje kontrolerowi fizyczną przeszkodę. Gracz o połowie szerokości 16 zatrzyma środek na x=174.

- Po sekundzie D środek Player zatrzymuje się na x=174, y=180; prawa krawędź dotyka x=190.
- Bez collidera na ścianie ten sam ruch dochodzi do x=220.
- Sprite ściany i collider mają niezależnie ustawione wymiary; sam Sprite nie blokuje ruchu.

2. Samodzielnie — Koło i skala wizualna

Samodzielnie zamień box gracza na CircleCollider2D(12) oraz ustaw skalę grafiki na (2,2). Ściana jest gotowa; D ma zatrzymać środek gracza na x=178.

- Przy tej samej ścianie Player zatrzymuje się na x=178.
- Po zmianie scale=(2,2) punkt zatrzymania nadal wynosi x=178.
- Jawna zmiana promienia na 20 zmienia punkt zatrzymania na x=170; sama skala tego nie robi.

3. Rozbudowa — Korytarz i granice świata

Gotowe są ruch, gracz i widoczne ściany korytarza. Samodzielnie dodaj ścianom collidery i ustaw granice świata 640×360. Sprawdź blokowanie ruchu i ślizganie po ścianie.

- Ruch w lewo zatrzymuje środek na x=192, w prawo na x=268.
- Podczas ruchu ukośnego wzdłuż ściany x nie przekracza ograniczenia, a y nadal rośnie.
- Na granicach świata środek koła pozostaje w x=12..628,y=12..348.
- Krok 0.1 s przy speed=600 nie przenosi gracza na drugą stronę ściany.

Lekcja 4 · Pierwszy przeciwnik i pościg

Czego się nauczysz

Uruchamiasz pościg za graczem przez komponent EnemyAI. Obliczasz kierunek do celu, korzystając z gotowego kontrolera; następnie samodzielnie piszesz pościg i ucieczkę.

Kierunek = pozycja gracza − pozycja wroga



Slime: x=100 Player: x=300

60 px/s × 1 s → Slime x=160

Wektor od przeciwnika do celu

Kierunek wyznaczamy jako pozycja celu minus pozycja przeciwnika. Dla przeciwnika (100,150) i celu (160,150) otrzymujemy (60,0), czyli w prawo.

Stała prędkość pościgu

Znormalizuj różnicę pozycji, zanim pomnożysz ją przez prędkość. Dzięki temu odległy cel nie powoduje szybszego ruchu. Obsłuż brak celu i odległość zero.

Decyzja i wykonanie ruchu

EnemyAI wybiera kierunek. Kontroler wykonuje ruch i uwzględnia przeszkody. Parametry pozwalają uzyskać różne zachowania tej samej klasy.

Gotowa baza i dalsza nauka

Gotowa baza: infrastruktura i mechaniki jeszcze niewyjaśnione w kursie. Edytuj wyłącznie miejsce TODO; pliki pomocnicze możesz obejrzeć.

Lekcja 4 · zadania

1. Z przykładem — Pościg za graczem

Gracz, Slime, cel i obliczenie dx/dy są gotowe. Dopisz jedno wywołanie move(dx,dy,speed) w EnemyAI. Wróg ma zacząć pościg z prędkością 60.

Zmiana, która uruchamia mechanikę:

EnemyAI.java

```
requireComponent(CharacterController2D.class).move(dx,dy,speed);
```

Cel, kierunek i kontroler są gotowe. Wywołanie move(dx, dy, speed) uruchamia pościg. Kierunek liczymy jako pozycja celu minus pozycja przeciwnika; kontroler normalizuje wektor.

- Po sekundzie Slime ma pozycję (160,100) na pustej scenie.
- Dla celu po przekątnej Slime pokonuje 60 px w sekundę.
- Jeżeli Slime i target są w tym samym punkcie, nie powstaje NaN ani niekontrolowany ruch.
- Zniszczenie celu lub target=null zatrzymuje AI bez błędu.

2. Samodzielnie — Dwa tempa pościgu

Slime i Bat mają własne prędkości 40 i 80 oraz referencję do gracza. Samodzielnie napisz onUpdate wspólnego EnemyAI: oblicz kierunek do celu i wywołaj kontroler.

- Na osobnych prostych odcinkach po 0.5 s Slime przebywa 20 px, a Bat 40 px.
- Zmiana name gracza nie zatrzymuje obu AI.
- Po przeniesieniu celu na drugą stronę przeciwnicy zmieniają kierunek następnego kroku.
- Oba sprite’y pozostają pionowo; wygląd może się odbijać przez flipX, rotation pozostaje 0.

3. Rozbudowa — Przeciwnik utrzymuje dystans

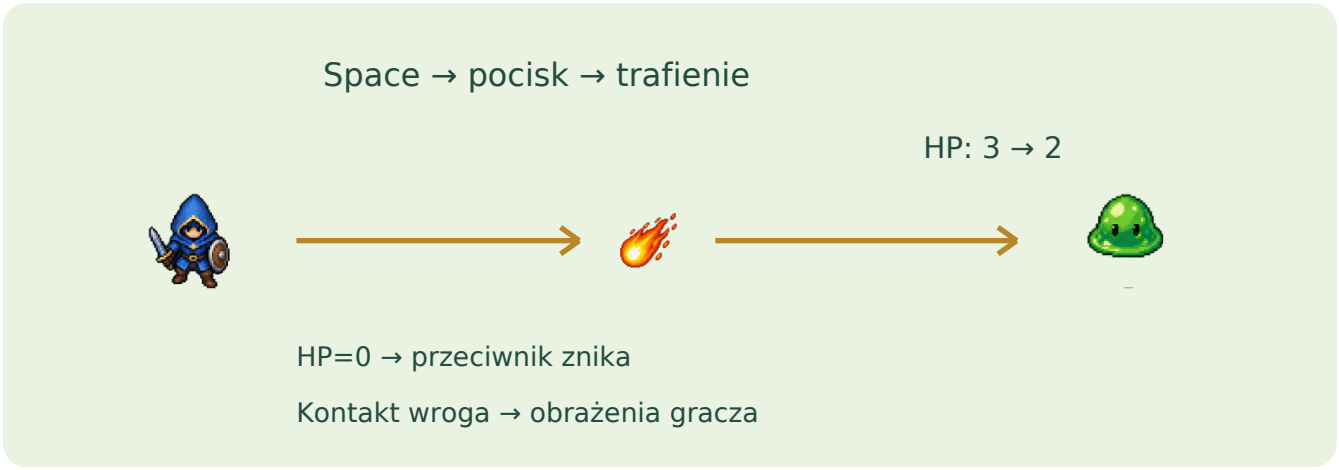
Masz gotową scenę, ruch gracza i komponent FleeAI. Samodzielnie napisz ucieczkę: przy odległości <120 oddalaj się z prędkością 60; poza zasięgiem stój.

- Dla target=(300,100),enemy=(250,100) po 0.5 s wróg znajduje się w (220,100).
- Gdy odległość wynosi 120 lub więcej, następny krok nie oddala przeciwnika.
- Przy tym samym położeniu celu i wroga stosujesz jawny kierunek awaryjny (1,0), bez NaN.
- FleeAI jednego wroga nie zmienia parametrów EnemyAI drugiego; do jednego obiektu podłączony jest tylko jeden aktywny autor kierunku ruchu.

Lekcja 5 · Pociski, HP i pierwsza pętla walki

Czego się nauczysz

Strzelasz zgodnie z ostatnim kierunkiem ruchu, odczytanym z CharacterController2D.getDirection(). Pocisk odbiera HP, przeciwnik znika po śmierci, a kontakt wroga odbiera HP graczowi. Gotowe komponenty później rozbudujesz.



Pocisk i HP tworzą walkę

Space tworzy pocisk w kierunku CharacterController2D.getDirection(). Kontakt pocisku z wrogiem wywołuje Health.damage(1): HP 3 → 2. Przy HP=0 wróg znika. Kontakt wroga odbiera HP graczowi. Health jest teraz gotowe; w lekcji 10 napiszesz i rozbudujesz ten komponent.

Pocisk: wygląd, lot i trafienie

Sprite rysuje pocisk, Projectile2D albo ProjectileMotion wykonuje lot, a ProjectileHit zmienia HP celu. Owner wskazuje obiekt strzelający. Każdy komponent ma osobną odpowiedzialność.

Jedno trafienie zużywa pocisk

Poprawny cel otrzymuje obrażenia. Pocisk ignoruje właściciela i sojuszników. Flaga zużycia chroni przed drugim trafieniem tego samego pocisku.

Gotowa baza i dalsza nauka

Gotowe Health, FireWeapon, ContactDamage i EnemyAI pozwalają grać. W tej lekcji zmieniasz trafienie; Health rozbudujesz w lekcji 10, Weapon w 11, AI w 14, a efekty w 19.

Sterowanie: WASD/strzałki; Space: strzał. Celuj zgodnie z ostatnim kierunkiem ruchu. RUN rozpoczyna świeżą próbę. Pokonaj wroga i sprawdź obrażenia gracza.

Lekcja 5 · zadania

1. Z przykładem — Prawdziwy lot pocisku

Lot pocisku, trigger, filtrowanie celu i Health są gotowe. Dopisz tylko hp.damage(damage) w ProjectileHit. Po trafieniu HP celu spadnie z 3 do 2; pocisk zniknie.

Zmiana, która uruchamia mechanikę:

ProjectileHit.java

```
hp.damage(damage);
```

Lot, trigger i Health są gotową bazą. hp.damage(damage) odejmuje zdrowie trafionemu celowi. Guard spent i usunięcie pocisku zapewniają jedno trafienie; owner chroni właściciela.

- Po 0.2 s pocisk jest w (140,100), cel nadal ma 3 HP.
- Po dotarciu odcinka ruchu do zakresu kontaktu cel ma 2 HP, a pocisk jest wycofany.
- Po kolejnych krokach cel nadal ma 2 HP; jeden pocisk nie nalicza trafienia wielokrotnie.
- Pozycja celu nie jest celem tweena, który odejmuje HP już w momencie wystrzału.
- Po uruchomieniu WASD/strzałki poruszają graczem, a Space tworzy pocisk w ostatnim kierunku ruchu.
- Trzy trafienia po 1 obrażeniu pokonują przeciwnika z HP=3; kontakt wroga odbiera HP graczowi z ochroną między trafieniami.

2. Samodzielnie — Właściciel i czas życia

Pocisk już lata i trafia. Samodzielnie zmień jego lifetime z 2 do 0.5 sekundy; zachowaj owner=Player i jedno trafienie. Sprawdź cel oraz zniknięcie pocisku.

- Zaraz po utworzeniu w środku Player jego Health nie maleje.
- Pocisk lecący obok wszystkich celów istnieje po 0.4 s i znika po 0.5 s.
- Dwa pociski utworzone w różnych chwilach mają własne czasy życia.
- Po wygaśnięciu pocisk nie rysuje się, nie bierze udziału w kontaktach i nie ma aktywnej aktualizacji.
- Po uruchomieniu WASD/strzałki poruszają graczem, a Space tworzy pocisk w ostatnim kierunku ruchu.
- Trzy trafienia po 1 obrażeniu pokonują przeciwnika z HP=3; kontakt wroga odbiera HP graczowi z ochroną między trafieniami.

3. Rozbudowa — Szybki pocisk i pierwszy cel

Trafienie i obiekty sceny są gotowe. Samodzielnie dodaj pociskowi Projectile2D(1,0,3000,1,player) i ustaw hitLayers=2. Ma trafić pierwszego wroga, ignorując sojusznika i właściciela.

- Przy kroku 0.1 s pocisk wykrywa cel r=8 między swoim startem a końcem kroku, mimo braku nakładania w pozycji końcowej.
- Przy dwóch celach na tej samej linii obrażenia otrzymuje tylko bliższy.
- Sojusznik znajdujący się przed Enemy nie zużywa pocisku ani nie traci HP.
- Nie powstają powtórne obrażenia po destroy; wynik trafienia jest taki sam przy krokach 0.1 s i 0.05 s dla tego nieruchomego układu.
- Po uruchomieniu WASD/strzałki poruszają graczem, a Space tworzy pocisk w ostatnim kierunku ruchu.
- Trzy trafienia po 1 obrażeniu pokonują przeciwnika z HP=3; kontakt wroga odbiera HP graczowi z ochroną między trafieniami.